

CPSC 2010: Introduction to Computer Science

Running Time of Programs April 13, 2026

Morning quiz

Instructions: Go to today's quiz on Canvas, and enter access code **show**



Investigate the running time of this program,
which prints $\sum_{i=0}^n i$.

```
n = ____;  
sum = 0;  
i = 1;  
while i <= n {  
    sum = sum + i;  
    i = i + 1;  
}  
print(sum)
```

compiler

```
loop: load i  
      sub n  
      skippos  
      jump next  
      jump done  
next: load sum  
      add i  
      store sum  
      load i  
      add one  
      store i  
      jump loop  
done: load sum  
      output  
      halt  
one: data 1  
sum: data 0  
i: data 1  
n: data ____
```

Feel free to talk with students sitting near you, then submit your answers.

Logistics

HW 7 due Thursday night

HW 8 will be released Thursday night, due Sunday, April 26, 11:59pm

All late work must be turned in by **May 4, 9am** (Yale policy)

Lowest homework grade will be dropped

1	loop: load i	9
2	sub n	10
3	skippos	11
	jump next	1
1	jump done	
	next: load sum	2
	add i	3
	store sum	4
	load i	5
	add one	6
	store i	7
	jump loop	8
2	done: load sum	
3	output	
4	halt	
	one: data 1	
	sum: data 0	
	i: data 1	
	n: data __	

Initial loop test (3 steps)

Loop iterations (11n steps)

Loop exit and output (4 steps)

Total runtime: $11n + 7$ steps

Computational complexity

Turing machines: what *can* be computed (given infinite resources)?

But just because something can be computed **in theory**, does not mean it can be computed **in practice**

- The universe might die before the computation finishes
- The computation may require more memory than atoms in the universe

Complexity theory: how many resources does a computation need?
i.e., what is **practically computable**?

1	loop: load i	9
2	sub n	10
3	skippos	11
	jump next	1
1	jump done	
	next: load sum	2
	add i	3
	store sum	4
	load i	5
	add one	6
	store i	7
	jump loop	8
2	done: load sum	
3	output	
4	halt	
	one: data 1	
	sum: data 0	
	i: data 1	
	n: data __	

Initial loop test (3 steps)

Loop iterations (11n steps)

Loop exit and output (4 steps)

Total runtime: $11n + 7$ steps

Notice:

- Runtime generally depends on *input size*
- The right way to think about “input size” is problem-dependent
 - Number of bits
 - Length of a list or array (related to number of bits)
 - Size of numerical input (related to number of bits)

```

sum = 0;
i = 1;
while i <= n {
    sum = sum + i;
    i = i + 1;
}
print(sum)

```

1 **load** i
2 **loop:** **sub** n
3 **skippos**
jump next
load sum
output
halt
next: **load** sum
add i
store sum
load i
add one
store i
jump loop
one: **data** 1
sum: **data** 0
i: **data** 1
n: **data** __

$10n + 6$ steps

9
10
1
2
3
4
5
6
7
8

1 **loop:** **load** i
2 **sub** n
3 **skippos**
jump next
jump done
next: **load** sum
add i
store sum
load i
add one
store i
jump loop
done: **load** sum
output
halt
one: **data** 1
sum: **data** 0
i: **data** 1
n: **data** __

$11n + 7$ steps

1
2
3
1
2
3
4

9
10
11
1
2
3
4
5
6
7
8

Initial loop test

Loop iterations

Loop exit and output

Computational complexity

We don't care about the specific constants—just how runtime grows with **n**

$$\begin{array}{c} 3n + 40 \\ 11n + 7 \\ 10n + 6 \end{array}$$

$$\longrightarrow O(n) \quad \text{“order } n\text{”}$$

$$\begin{array}{c} 5n^2 + 10n \\ 100n^2 + 3 \\ 3n^2 + 6n + 2 \end{array}$$

$$\longrightarrow O(n^2) \quad \text{“order } n^2\text{”}$$

Computational complexity

$f \in O(g)$ means

“eventually, some multiple of g is bigger than f ”

$O(g)$ is the **set of functions** f for which

there is some N and c such that for all $n > N$,

$$f(n) \leq cg(n)$$

Abuse of notation: $f = O(g)$ “ f is big-Oh of g ”

Computational complexity

We don't care about the specific constants—just how runtime grows with **n**

$$\begin{array}{c} 3n + 40 \\ 11n + 7 \\ 10n + 6 \end{array}$$



$O(n)$ “order n”

$$11n + 7 \leq 12n \text{ for } n > 7$$

$$3n + 40 \leq 4n \text{ for } n > 40$$

$$10n + 6 \leq 11n \text{ for } n > 6$$

$$\begin{array}{c} 5n^2 + 10n \\ 100n^2 + 3 \\ 3n^2 + 6n + 2 \end{array}$$

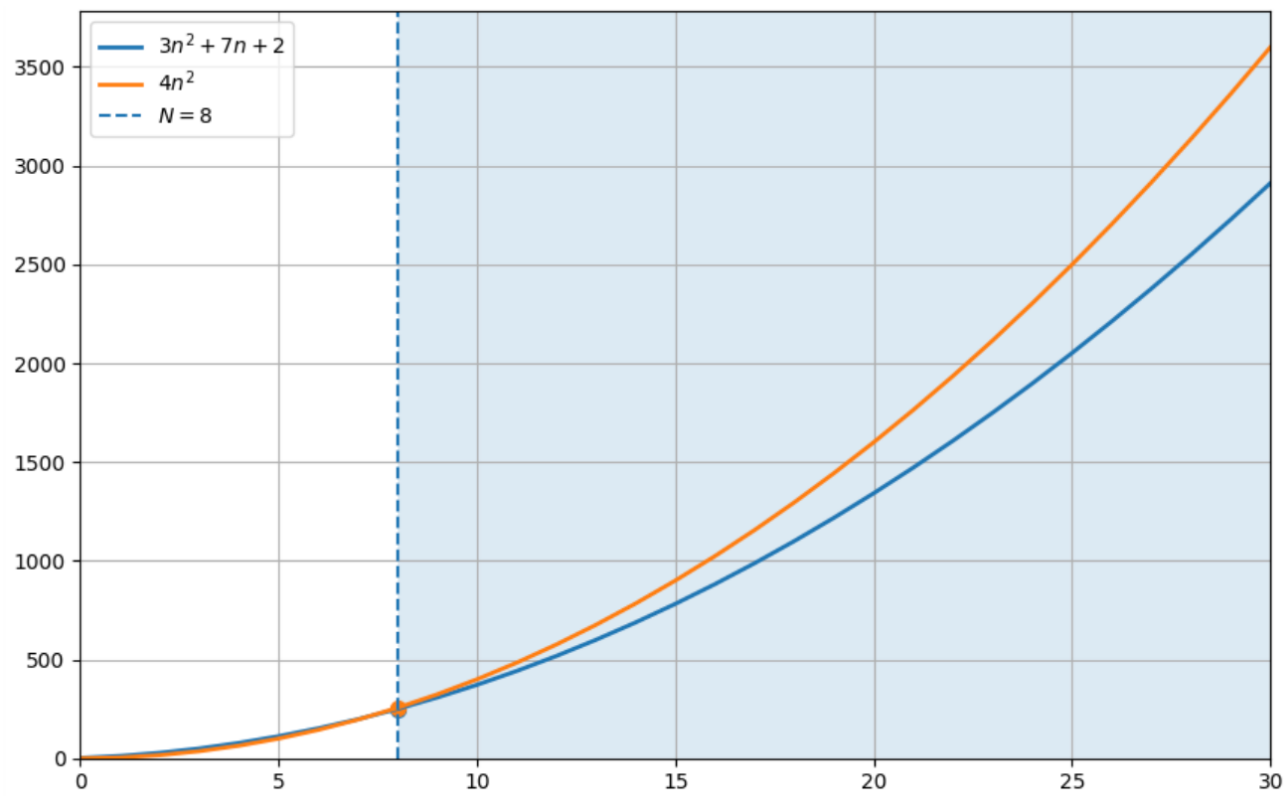


$O(n^2)$ “order n^2 ”

$$5n^2 + 10n \leq 6n^2 \text{ for } n > 10$$

$$100n^2 + 40 \leq 101n^2 \text{ for } n > 7$$

$$3n^2 + 7n + 2 \leq 4n^2 \text{ for } n > 8$$



$$3n^2 + 7n + 2 \in O(n^2)$$

Computational complexity

Note: we choose the *simplest* expression to put inside the big O

$$\begin{array}{c} 3n + 40 \\ 11n + 7 \\ 10n + 6 \end{array}$$

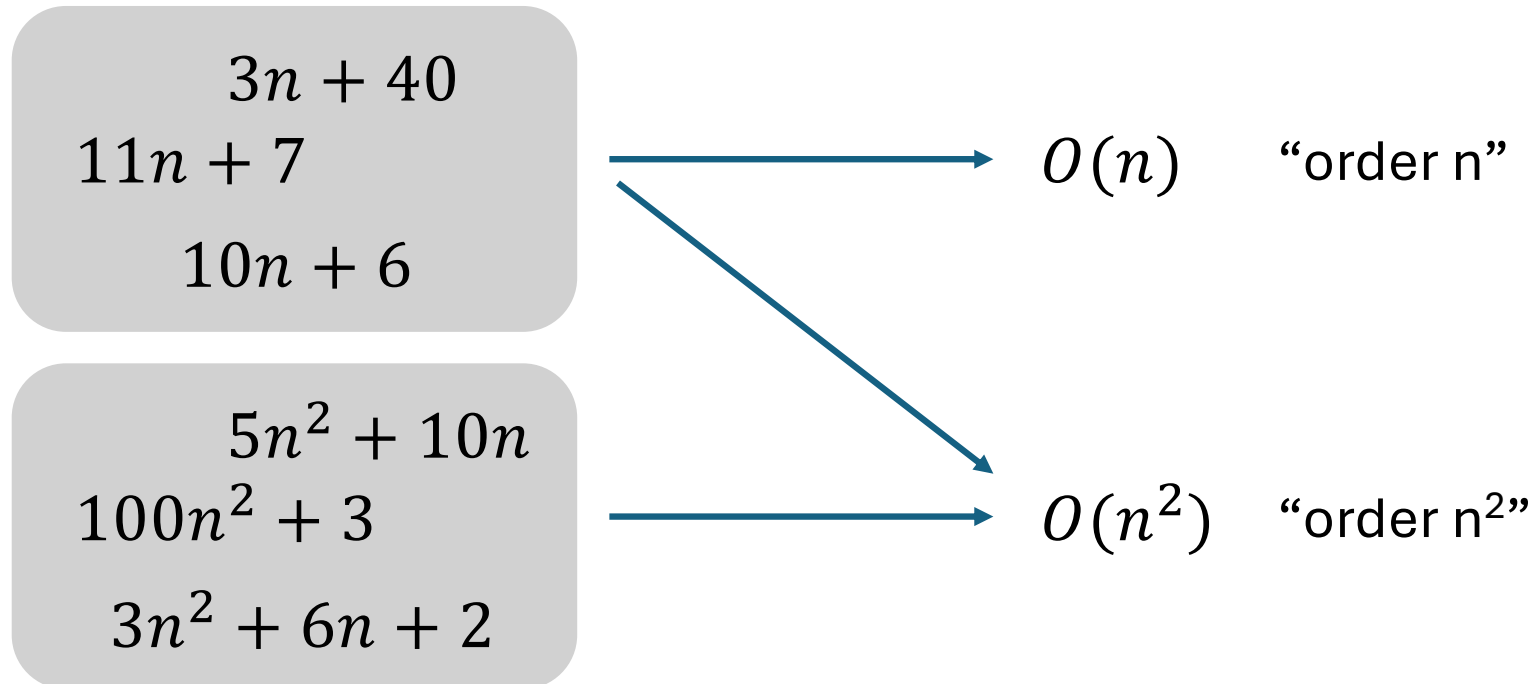
$$\longrightarrow O(n + 2) \text{ “order } n\text{”}$$

$$\begin{array}{c} 5n^2 + 10n \\ 100n^2 + 3 \\ 3n^2 + 6n + 2 \end{array}$$

$$\longrightarrow O(n^2) \text{ “order } n^2\text{”}$$

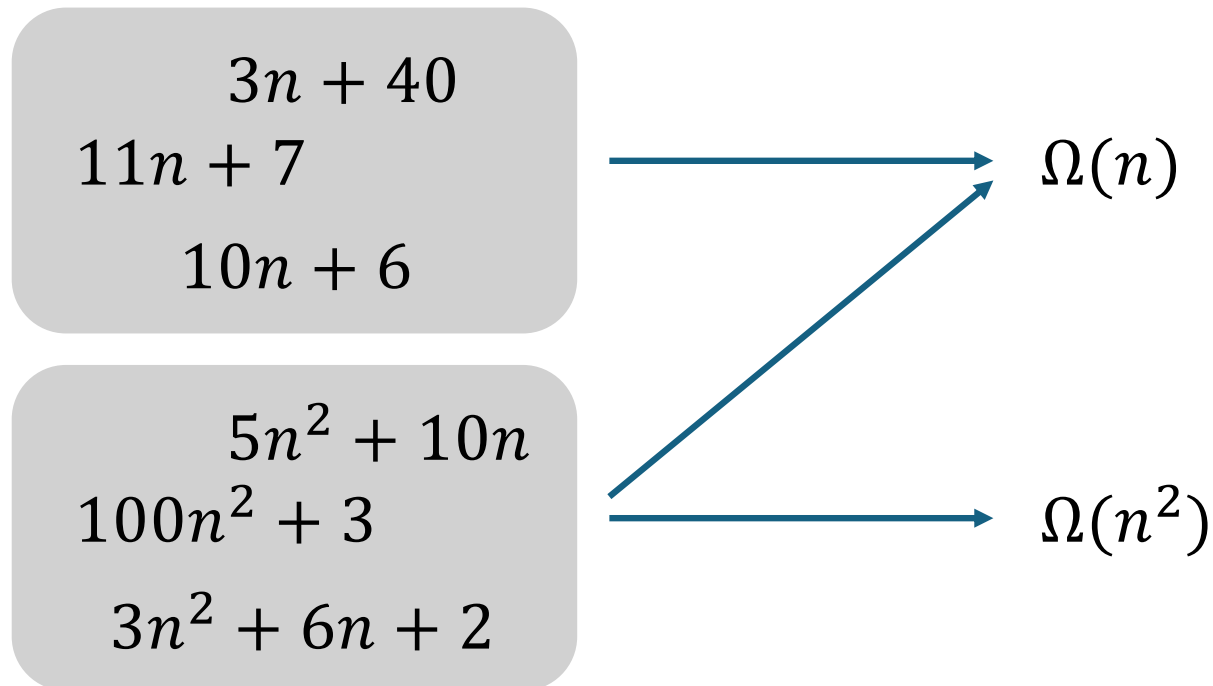
Computational complexity

Note: we choose the *simplest* expression to put inside the big O



Computational complexity

If $g \in O(f)$, we say $f \in \Omega(g)$



Computational complexity

If $f \in O(g)$, and $f \in \Omega(g)$, then $f \in \Theta(g)$

$$\begin{array}{c} 3n + 40 \\ 11n + 7 \\ 10n + 6 \end{array}$$

$$\longrightarrow \Theta(n)$$

$$\begin{array}{c} 5n^2 + 10n \\ 100n^2 + 3 \\ 3n^2 + 6n + 2 \end{array}$$

$$\longrightarrow \Theta(n^2)$$

Computational complexity

Let $T(n)$ be the number of steps taken by a given algorithm on an input of size n

If $T(n) = O(1)$: constant

If $T(n) = O(\log n)$: logarithmic

If $T(n) = O(n)$: linear

If $T(n) = O(n \log n)$: “linearithmic”

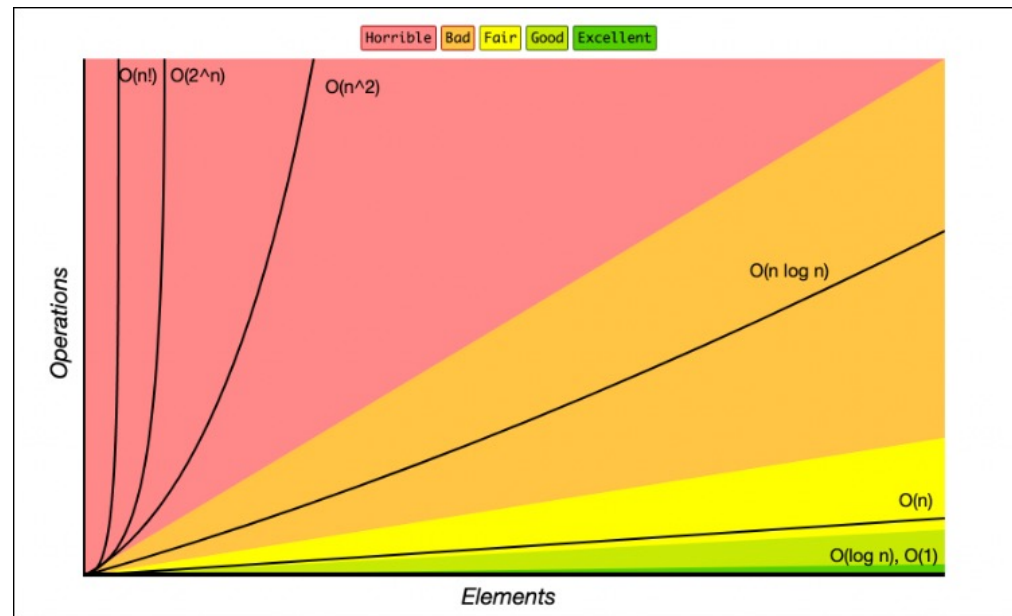
If $T(n) = O(n^2)$: quadratic

If $T(n) = O(n^3)$: cubic

If $T(n) = O(n^k)$: polynomial

If $T(n) = O(b^n)$: exponential

If $T(n) = O((cn)!)$: factorial



Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

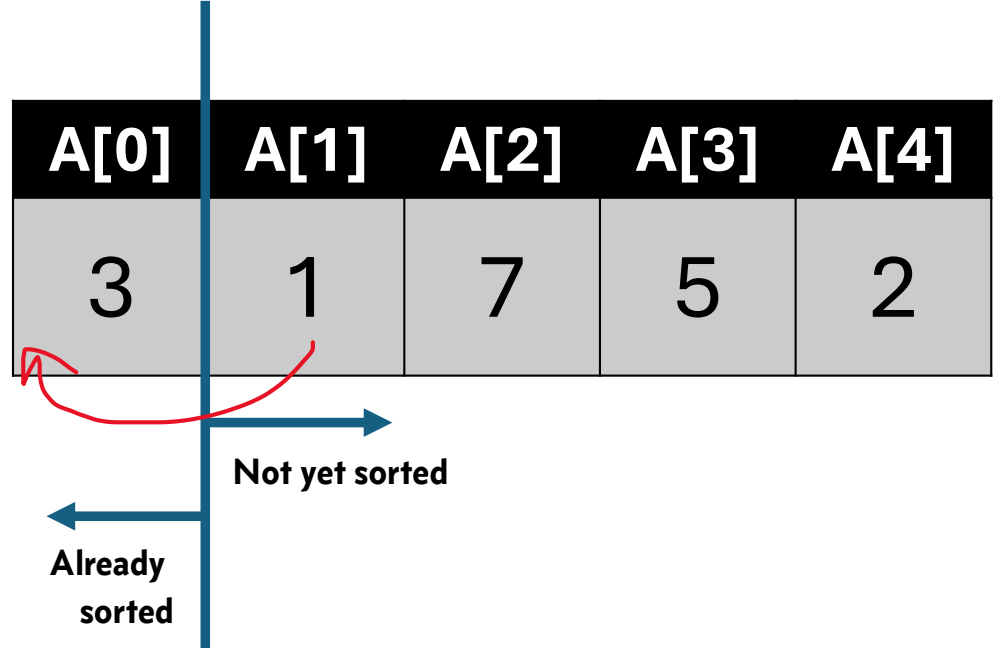
A[0]	A[1]	A[2]	A[3]	A[4]
3	1	7	5	2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

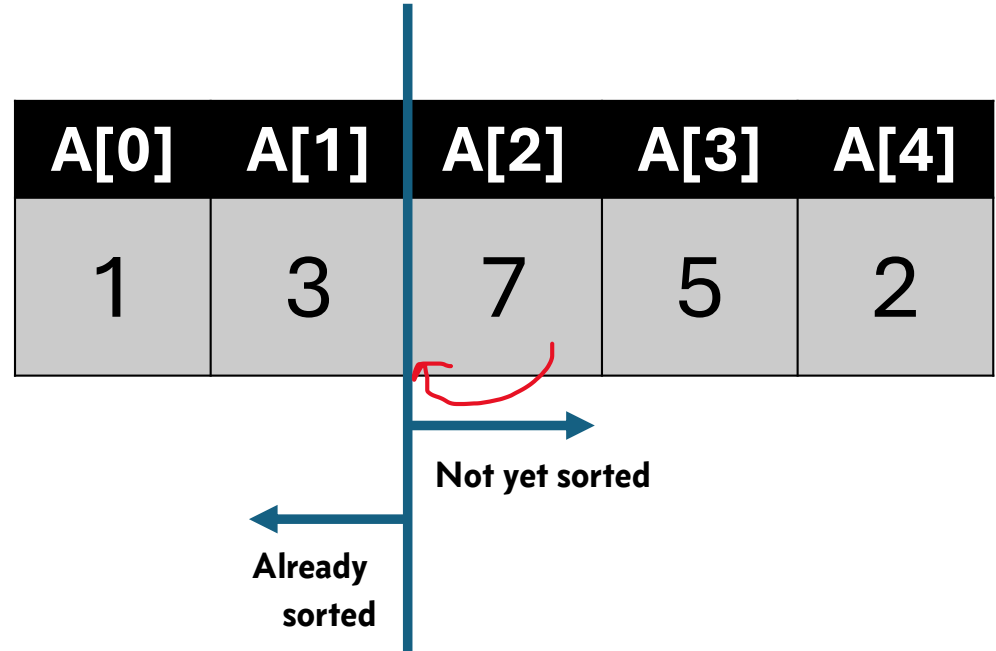


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

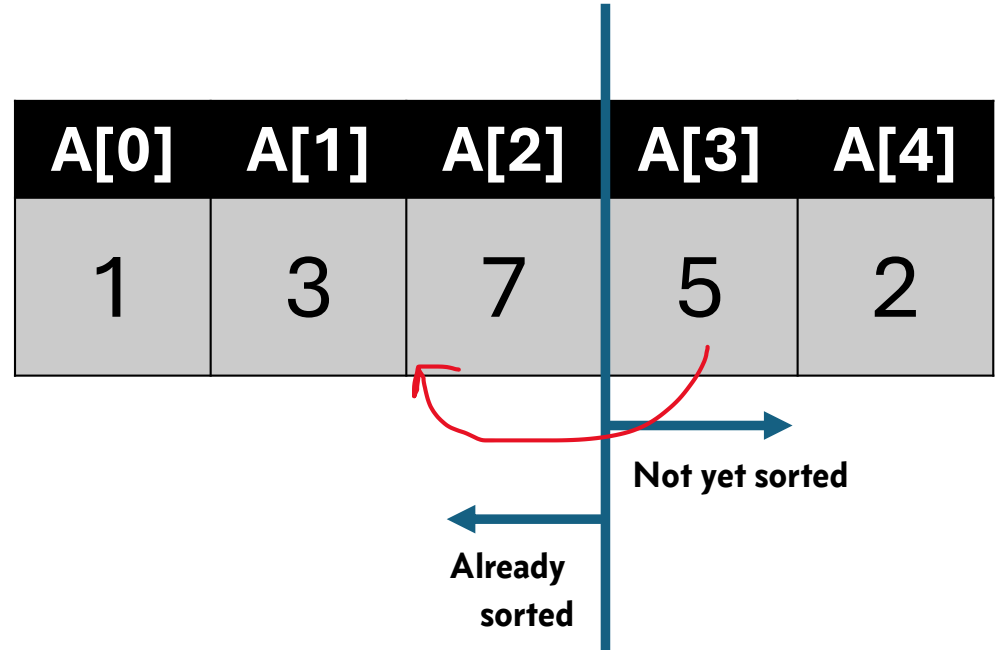


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

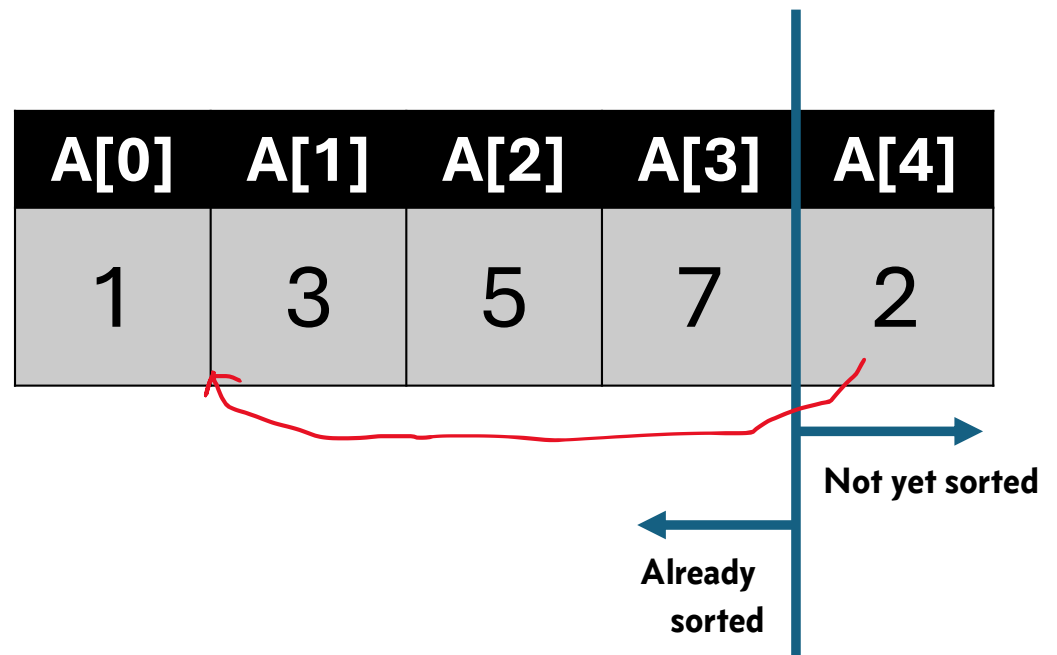


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	2	3	5	7

←
Already
sorted

Not

Computational complexity

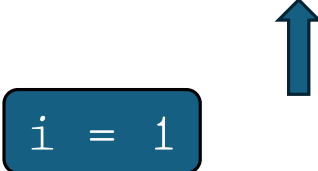
Time complexity: how much **time** does a computation take?

An example: sorting an array

➔ **for** $i \leftarrow 1$ **to** $\text{length}(A)$:
 $x \leftarrow A[i]$
 $j \leftarrow i$
 while $j > 0$ **and** $A[j-1] > x$:
 $A[j] \leftarrow A[j - 1]$
 $j \leftarrow j - 1$
 $A[j] \leftarrow x$

A[0]	A[1]	A[2]	A[3]	A[4]
3	1	7	5	2

$i = 1$



Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

➔

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
3	1	7	5	2



i = 1

x = 1

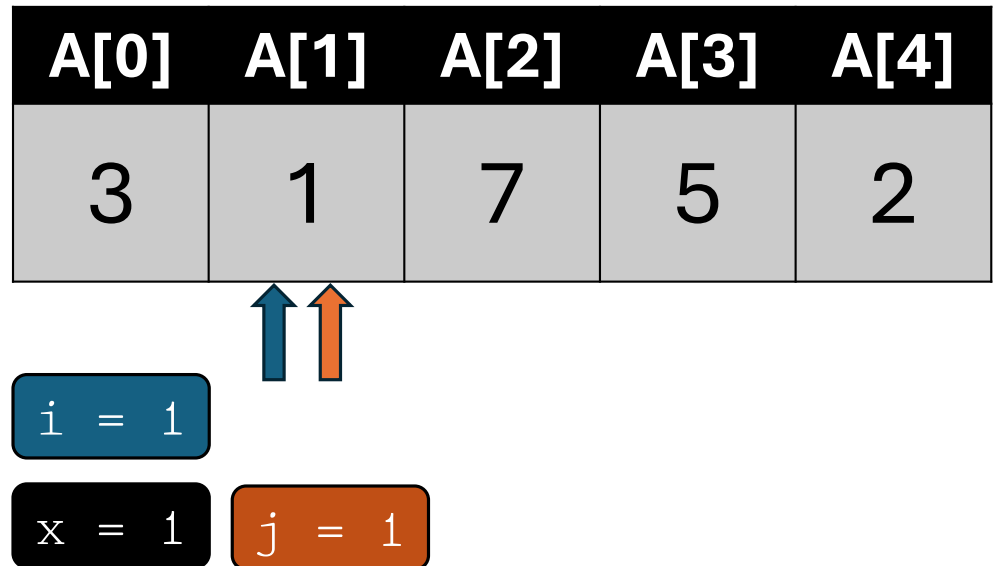
Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

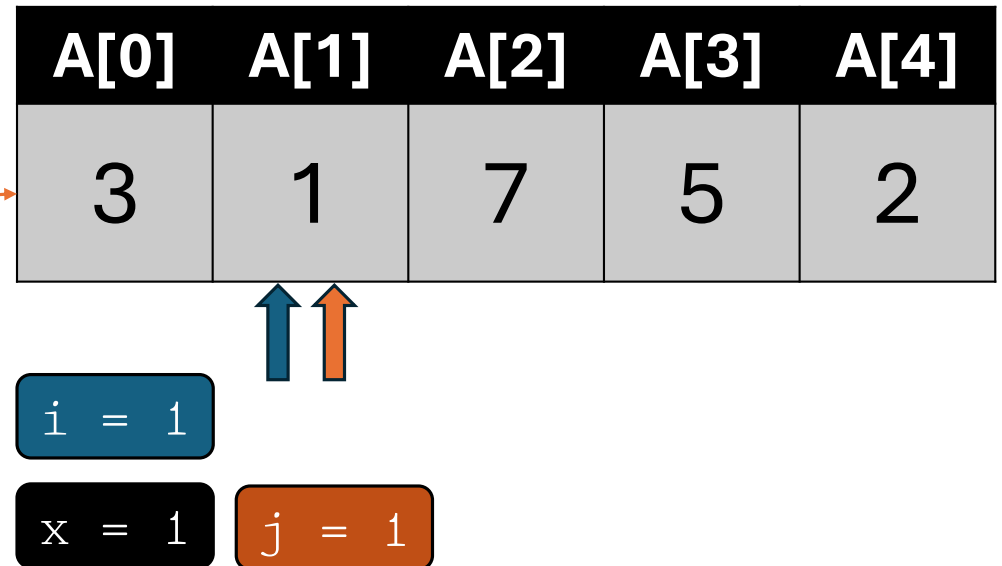


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

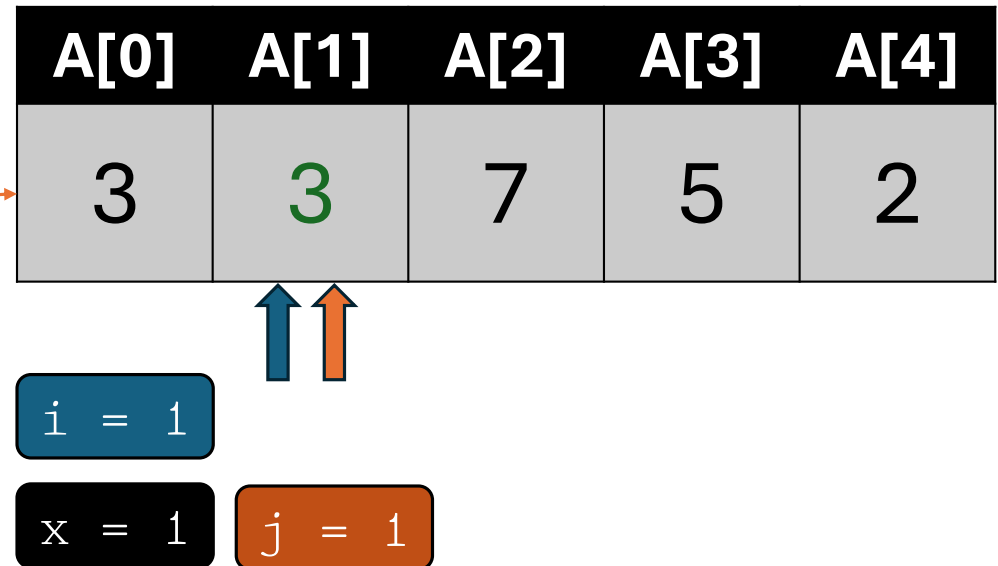


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



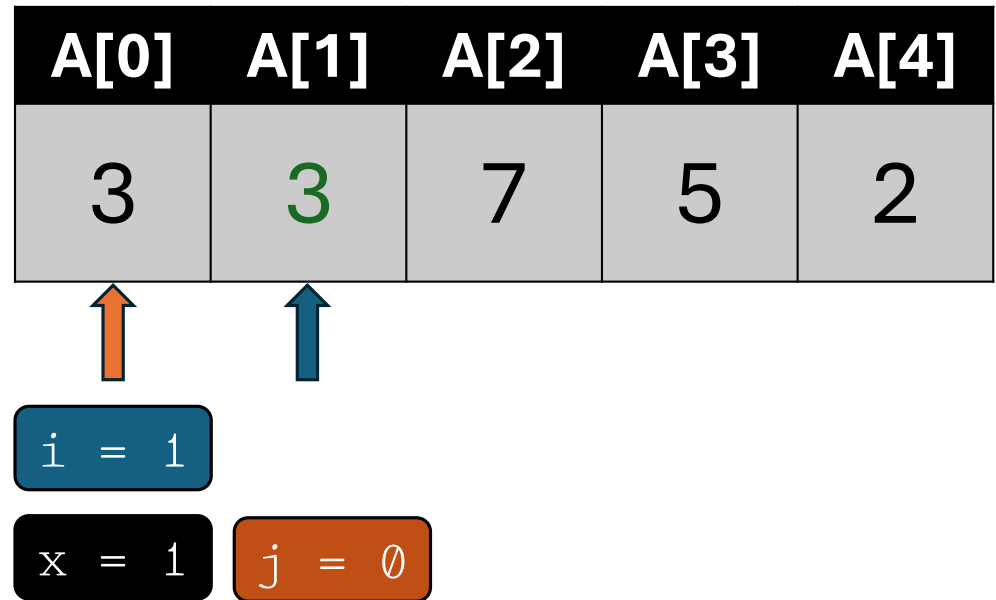
Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

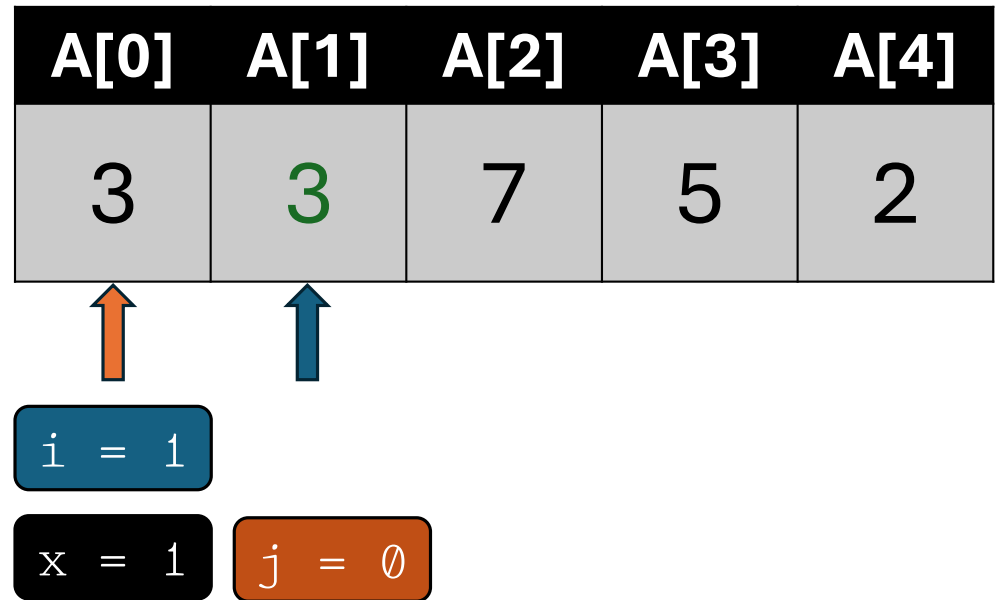


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

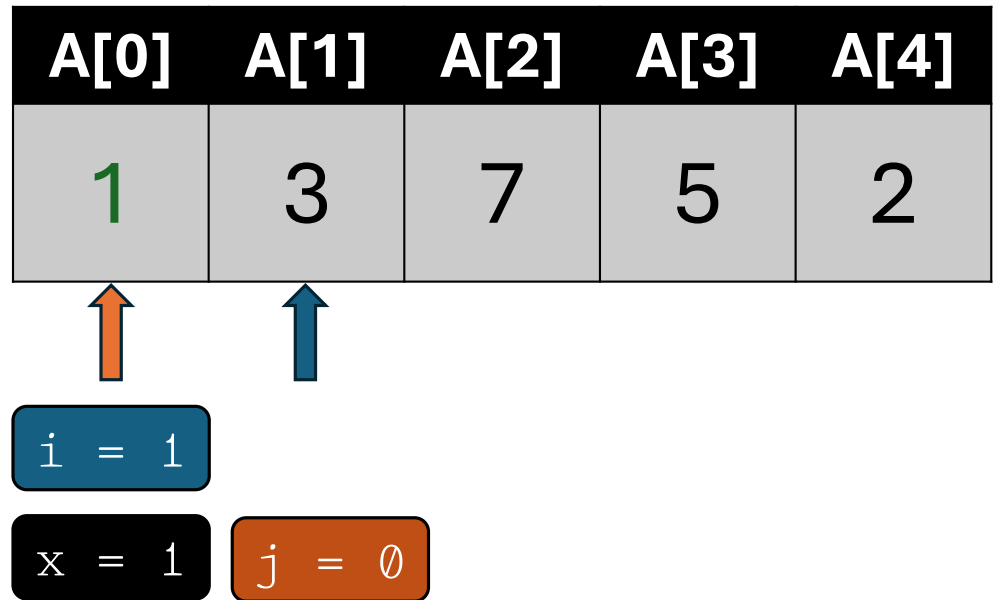


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

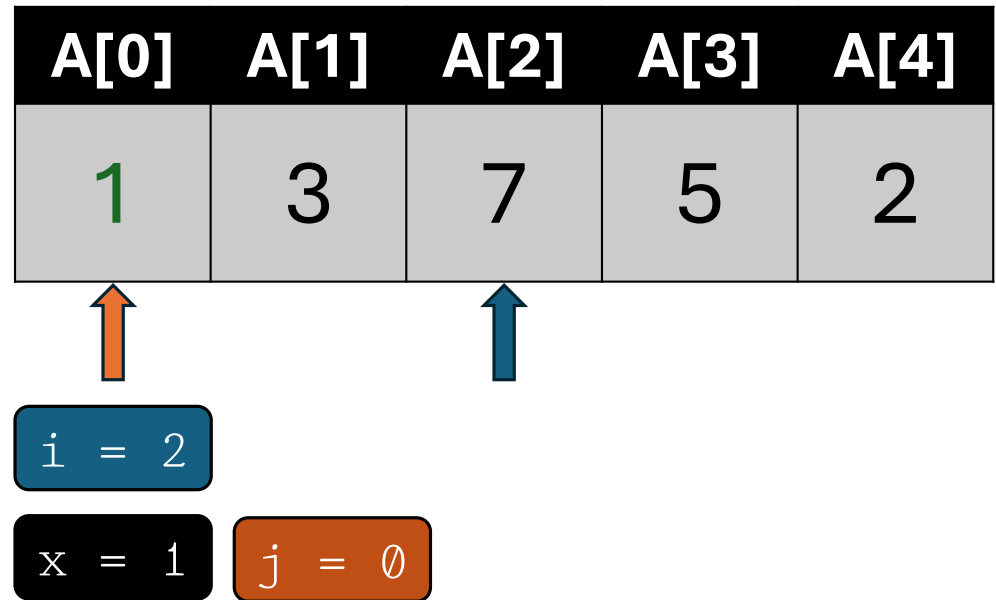


Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
➔ for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



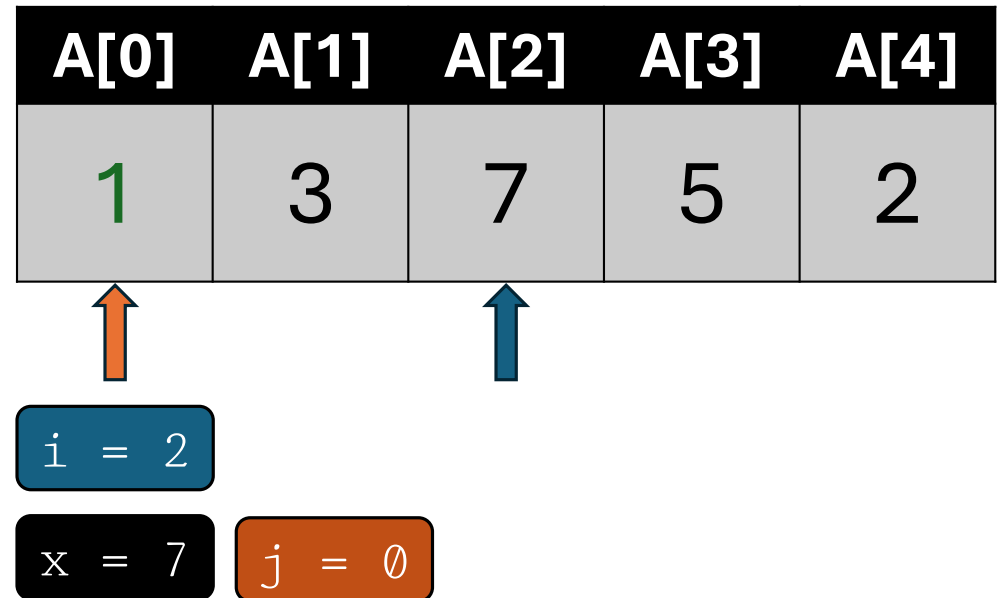
Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

➔

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

➔

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	5	2



i = 2

x = 7

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	5	2



i = 2

x = 7

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	5	2



i = 2

x = 7

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	5	2



i = 3

x = 5

j = 3

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	5	2



i = 3

x = 5

j = 3

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	7	2



i = 3

x = 5

j = 3

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	7	2



i = 3

x = 5

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	7	7	2



i = 3

x = 5

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	7	2



i = 3

x = 5

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	7	2



i = 4

x = 2

j = 4

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	7	2



i = 4

x = 2

j = 4

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	7	7



i = 4

x = 2

j = 4

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	7	7



i = 4

x = 2

j = 3

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	7	7



i = 4

x = 2

j = 3

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	5	7



i = 4

x = 2

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	3	5	5	7



i = 4

x = 2

j = 2

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	3	5	7



i = 4

x = 2

j = 1

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

→

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	3	3	5	7



i = 4

x = 2

j = 1

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```



A[0]	A[1]	A[2]	A[3]	A[4]
1	2	3	5	7



i = 4

x = 2

j = 1

Computational complexity

Time complexity: how much **time** does a computation take?

An example: sorting an array

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

A[0]	A[1]	A[2]	A[3]	A[4]
1	2	3	5	7

Computational complexity

Time complexity: how much **time** does a computation take?

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

How long does it take to run this algorithm?

Computational complexity

Time complexity: how much **time** does a computation take?

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

How many steps does this algorithm take?

A[0]	A[1]	A[2]	A[3]	A[4]	BEST CASE
1	2	3	5	7	

A[0]	A[1]	A[2]	A[3]	A[4]	WORST CASE
9	6	5	3	1	

Computational complexity

Time complexity: how much **time** does a computation take?

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

Asymptotic analysis:

Let **T(n)** be the number of steps taken by the

- **worst-case** input of size **n**.
- **best-case** input of size **n**.
- **average** input of size **n** (over some *distribution*)

Computational complexity

Time complexity: how much **time** does a computation take?

```
for i <- 1 to length(A):  
  x <- A[i]  
  j <- i  
  while j > 0 and A[j-1] > x:  
    A[j] <- A[j - 1]  
    j <- j - 1  
  A[j] <- x
```

Asymptotic analysis:

Let **T(n)** be the number of steps taken by the

- **worst-case** input of size **n** $\Rightarrow T(n) \in \Theta(n^2)$
- **best-case** input of size **n**. $\Rightarrow T(n) \in \Theta(n)$
- **average** input of size **n** (over some *distribution*). $\Rightarrow T(n) \in \Theta(n^2)$

Computational complexity

Question to think about...

Comparing each element to each other element is $O(n^2)$ comparisons

Is quadratic-time the best we can do, when sorting an array?